TOKEN ARCADE v0 - METROPOLIS

BUILD HANDOFF / SCOPE FREEZE

Project: Token Arcade
Release: v0 - Metropolis
Revision: 8 Oct 2026 alignment pass
North Star: The Internet Is Fun.
Product: Curated web-browser mini-game arcade for onchain communities.
Initial deployment: Monad
Architecture: Do not unnecessarily couple the product or games to Monad.
Deadline priority: Working, beautiful, comprehensible demo > theoretical completeness.

---

0. CODEX DIRECTIVE

You are working in the actual Token Arcade repository.

Treat this document as the source of truth for **Token Arcade v0 / Metropolis**.

Use parallel agents/branches where useful.

Before coding:

1. Inspect the entire repository.
2. Identify existing games, frameworks, assets, contracts and partial implementations.
3. Preserve working code.
4. Document architecture before making sweeping changes.
5. Reuse existing systems instead of rebuilding them unnecessarily.
6. Divide work into independent branches/workstreams.
7. Maintain a single integration branch.
8. Continuously run/build/test the integrated application.
9. Do not implement roadmap features unless they are necessary to avoid architectural dead ends.
10. If a roadmap feature can be enabled later through an interface, create the interface-not the feature.

The objective is not to produce ten prototypes.

The objective is to leave the repository with **one coherent Token Arcade product that works end-to-end.**

---

1. PRODUCT THESIS

Token Arcade is a browser arcade for internet communities.

A visitor should be able to land on the site and immediately understand:

> There are fun games here.
> I can play them immediately.
> If I create an account, my arcade identity persists.
> Some games have competitive modes.
> Some games have their own experimental economies.
> Some games are connected to existing internet communities.
> Those communities can host competitions and reward their players.

Crypto infrastructure should enable the product.

Crypto infrastructure should **not dominate the interface.**

The user should experience:

game -> fun -> identity -> competition -> community -> rewards

rather than:

wallet -> token -> contract -> transaction -> game

---

2. V0 PRODUCT POSITIONING

Token Arcade v0 is a:

curated browser-based mini-game arcade with optional onchain game economies and community competitions.

V0 launches on Monad for Metropolis.

V0 is curated.

Users cannot permissionlessly launch games yet.

The v0 creator lifecycle is:

```
SUBMIT
  ->
ADMIN REVIEW
  ->
APPROVE
  ->
DEVELOPER DEPLOY
  ->
PUBLISHED
```

Approval and deployment are separate actions.

- The **developer/creator** submits the Cartridge plus launch configuration.
- The **Token Arcade admin** reviews and approves or rejects it.
- After approval, the **developer initiates Deploy**.
- The public Arcade renders only deployed/published Registry entries.

For Metropolis, the founder may control both the developer and admin identities for the demo, but the product model must keep the roles separate.

The initial legacy cabinets can be seeded into the Registry as already approved/deployed. The Metropolis demo should prove the real pipeline by taking **Skrumpey Slide** from submitted -> approved -> developer-deployed -> live in the Arcade.

V0 does not require the developer wallet itself to have unrestricted protocol authority. Where the economic source of truth keeps an onchain launch permission platform-controlled, the approved developer's Deploy action may invoke the approved platform deployment path. The UX lifecycle remains Submit -> Approve -> Deploy.

---

3. TERMINOLOGY - SOURCE OF TRUTH

TOKEN ARCADE

Token Arcade / TA means the platform, application, brand and protocol infrastructure.

Do not use "Token A" publicly during v0 unless discussing historical/internal architecture.

There is **no Token Arcade token in v0.**

TOKEN ARCADE TREASURY

Protocol revenue that may eventually have flowed toward a hypothetical Token Arcade token instead flows into the:

Token Arcade Treasury

This treasury may hold MON, stablecoins or other supported assets.

There is:

- no promise of a future `$TA`
- no claim that treasury assets belong to hypothetical future token holders
- no token launch required for Metropolis

A future protocol token remains an optional design decision.

MINIGAME

A playable game experience inside Token Arcade.

A Minigame may be implemented from a reusable Gameplay Skeleton, but the player competes in a specific Cabinet/Deployment.

A Minigame does **not** need to have a token.

GAMEPLAY SKELETON

Reusable gameplay mechanics, simulation rules, controls, scoring logic, and technical structure that can underpin multiple separately deployed games.

A Skeleton is **not** itself a public leaderboard or economy.

Example:

```
Buy Bounce source mechanics -> reusable vertical-jumper Skeleton
```

That Skeleton can later power multiple independently branded cabinets.

CABINET / DEPLOYMENT

The public game deployment that appears in Token Arcade.

A Cabinet has its own:

- stable `gameId`
- public name/branding
- Community Asset association, if any
- optional MiniCoin/economy configuration
- leaderboards
- Raid history
- publication state

Leaderboards belong to Cabinet deployments, not Skeletons.

If two communities deploy separate games from the same Skeleton, they do not share a leaderboard by default. The Player Passport may still aggregate the player's history across both cabinets.

SKIN / THEME LAYER

Presentation that can change without silently changing competitive gameplay: characters, art, backgrounds, UI theme, music, SFX, copy, and community memetics.

A reskin can create a separately branded Cabinet while reusing the same underlying Skeleton.

If a change modifies the scored gameplay path, it is a new gameplay/Cartridge version rather than a cosmetic reskin.

MINICOIN

Internal architecture name:

Token B

Public/product name:

Minicoin

A Minicoin is an **optional game-specific fungible token** associated with a Minigame.

Examples:

```
Skrumpey Slide -> optional Skrumpey Slide Minicoin
```

A game can exist without a Minicoin.

A Minicoin can eventually implement different economic strategies.

Do not hardcode one universal Minicoin tokenomic model.

Potential future Minicoin behavior includes:

- game reward token
- game economy
- fee/tax token
- reward redistribution
- treasury accumulation
- liquidity hooks
- Token C accumulation
- player rewards
- holder rewards

For Metropolis, prove **deployment and association**, not the final tokenomic theory.

COMMUNITY ASSET

Historical/internal architecture name:

Token C

Preferred terminology:

Community Asset

A Community Asset represents the established community that a Minigame is associated with.

A Community Asset can be:

- ERC-20 token
- ERC-721 NFT collection
- ERC-1155 collection
- community treasury
- potentially other supported community primitives later

Therefore:

Token C does NOT mean "community coin."

It means:

the canonical asset/community identity that Token Arcade is supporting.

Example:

Skrumpeys NFT collection = Community Asset for Skrumpey Slide.

COMMUNITY STRATEGY

The Community Strategy defines how a game interacts with its Community Asset.

Examples:

ERC20 Community Strategy

May eventually define:

- liquidity relationship
- reward routing
- fee routing
- Token C accumulation
- holder eligibility
- hooks
- buybacks

- treasury routing

NFT Community Strategy

May define:

- NFT holder verification
- NFT profile pictures
- holder cosmetics
- holder rewards
- NFT raid prizes
- prize vaults
- community treasury routing

The Minicoin itself is **not** the strategy.

Keep these systems separate.

PLAYER PASSPORT

Persistent Token Arcade player identity.

The Passport eventually represents:

- games played
- verified runs
- high scores
- raids
- wins
- achievements
- rewards
- communities supported
- cosmetic identity
- selected avatar/PFP
- Arcade Score
- owned/unlocked cosmetics

V0 can implement only the data necessary for the demo while preserving this model.

RANKED RUN

The user-facing name for authenticated competitive play.

A Ranked Run is authenticated competitive play. Paid entry is a later/legal-review feature and is off in Metropolis v0 unless the Protocol Source of Truth is explicitly revised.

A Ranked Run:

- requires Player Account
- uses defined game conditions
- creates a verifiable result
- updates leaderboard/history
- may update Player Passport
- may distribute rewards

Avoid calling the mode merely “Pay to Play.”

Payment is a mechanic.

Ranked Run is the product mode.

COMMUNITY RAID

A limited-time competition attached to:

- a game
- rules
- objective
- course/seed if relevant
- attempts
- leaderboard
- Community Asset
- optional entry cost
- optional Minicoin
- optional rewards

Example:

Skrumpey Slide Community Raid

Objective:

`TOTAL_SCORE`

Attempts:

`3`

Community:

`Skrumpeys`

Prize:

`Skrumpey #XXXX`

COMMUNITY SPRINT - ROADMAP

Not v0.

Longer-duration community competition where individual players contribute toward a collective community/clan score.

Think:

raid x season x clan war

Possible future metrics:

- collective player score
- participation
- volume
- holders
- game activity
- Minicoin activity
- Token C activity

---

4. V0 USERS / ACCOUNT MODEL

There are four v0 roles. One human may hold more than one role in the Metropolis demo, but the product logic must keep the permissions distinct.

A. GUEST

No login.

Guest can:

- browse arcade
- open game pages
- play Free Play
- replay indefinitely
- view public leaderboards
- view public raids
- view game/community information

Guest cannot:

- enter Ranked Runs
- enter gated Community Raids
- receive persistent rewards
- maintain persistent Passport
- use holder-gated rewards

Guest progress can remain local/session-only.

The first interaction with Token Arcade should **not require wallet connection.**

B. PLAYER ACCOUNT

Created/authenticated through Privy.

Player Account contains:

- Privy identity
- embedded arcade wallet
- Token Arcade player ID
- Player Passport
- scores/history
- raid history
- rewards
- linked ownership wallet(s)

Player can:

- Free Play
- Ranked Run
- enter Community Raids
- maintain persistent scores
- appear on Cabinet leaderboards
- earn rewards
- select profile identity
- link an external wallet
- prove Community Asset ownership

C. CREATOR / GAME DEVELOPER

Creator can:

- submit a Token Arcade Cartridge
- provide metadata/artwork
- select supported game modes/objectives
- associate a Community Asset
- configure an optional MiniCoin/economy request
- select approved Strategy options exposed by the platform
- view submission status
- respond to validation/review issues
- initiate **Deploy** after admin approval

- later request/enable an economy for an existing game-only Cabinet without changing its `gameId`

Creator cannot approve their own submission in the v0 permission model unless they also hold the explicit Admin role.

D. TOKEN ARCADE ADMIN

V0 uses an allowlisted Token Arcade admin as curator/reviewer.

Admin can:

- inspect submitted Cartridges/manifests
- run or review technical validation
- approve/reject submissions
- approve Community Asset/economy configuration
- manage Registry status
- pause/unpublish for safety
- create or administer platform/community Raid configuration where required
- monitor activity and settlement

The admin does **not** replace the creator's final Deploy action in the normal v0 product flow.

For Metropolis, **Andrew / Token Arcade operator may act as both Creator and Admin** to demonstrate the two states without building complex organization/RBAC infrastructure.

---

5. OWNERSHIP WALLET / DELEGATION MODEL

Players should NEVER be expected to move valuable Community Assets into their embedded Privy arcade wallet merely to use Token Arcade.

Separate:

Arcade Wallet

Used for:

- game entries
- arcade transactions
- rewards
- Passport activity

Ownership / Vault Wallet

May contain:

- Community ERC-20s
- Skrumpeys
- NFTs
- valuable holdings

Player should be able to prove:

> My arcade identity is authorized to use the community rights associated with this ownership wallet.

V0 ownership solution should support the safest practical combination of:

1. connected external wallet
2. signed wallet association
3. optional Delegate integration where supported
4. no asset transfer requirement

Create an abstraction such as:

`OwnershipResolver`

Possible implementations:

`DirectWalletOwnershipResolver`

`DelegatedOwnershipResolver`

The rest of Token Arcade should call:

`isEligible(player, community)`

and should not care how eligibility was proven.

---

6. GAME CARTRIDGE MODEL

Every game integrated into Token Arcade should behave like a cartridge.

The host application should be able to provide configuration and receive structured results.

Conceptually:

```
interface TokenArcadeGame {
  mount(config): void
  start(): void
  pause(): void
  resume(): void
  restart(): void
```

```
  destroy(): void
  onRunComplete(callback): void
}
```

Run result should resemble:

```
{
  gameId,
  gameVersion,
  runId,

  mode,
  raidId?,

  seed?,
  objective,

  primaryMetric,
  secondaryMetrics?,

  result,

  startedAt,
  finishedAt
}
```

Games should not directly contain:

- Privy logic
- Monad logic
- Minicoin contracts
- raid contracts
- wallet logic
- NFT ownership logic

The arcade hosts the game.

The game returns the result.

The Token Arcade platform decides what happens next.

Deployment identity rule: the same underlying Gameplay Skeleton may power many Cabinets. Each Cabinet/deployment gets its own `gameId`, leaderboard namespace, Raid history, community association, and optional economy. Skeleton reuse does not merge scores by default.

---

7. V0 GAMES

The initial Arcade should be populated through Registry/publication state, not hardcoded homepage cards.

INITIAL LEGACY CABINETS

eMONAD CABINET - SOURCE SKELETON: BUY BOUNCE

Public identity:

- an eMONAD-branded game/cabinet
- public title/ticker can be finalized separately
- **"Buy Bounce" is the source implementation / internal Skeleton reference, not required public branding**

Community Asset:

`eMONAD` - founder-supplied contract reference: `0x81a224f8a62f52bde942dbf23a56df77a10b7777`

Before any economic deployment, verify current chain/address facts in the protocol implementation pass.

Genre:

endless vertical platformer / endless jumper

Target:

- Cartridge integration
- Free Play
- Ranked-ready
- structured RunResult
- eMONAD visual/community reskin
- planned optional MiniCoin/economy connection after the dedicated Protocol attack; no MiniCoin logic inside this Cartridge run

Reskin rule:

Presentation may change freely within the declared skin/theme surfaces. Preserve the competitive/scored path unless deliberately creating a new gameplay version.

A future community can reuse the same Skeleton for a completely different Cabinet with its own `gameId`, branding, leaderboard, raids, and economy.

TWO PLACES AT ONCE

Community Asset:

Skrumpeys ERC-721 collection

Do **not** broadly reskin the game for v0.

Preserve the existing world/game identity. Skrumpey content may be additive later, for example:

- selected enemies represented by particular Skrumpeys
- trait-aware content/progression
- themed collectibles or levels
- owned Skrumpey cosmetic/avatar usage

Genre:

puzzle-platformer / precision platformer

Primary metric:

- completion time

Secondary:

- stars/collectibles

Target:

- Cartridge integration
- Free Play
- Ranked-ready
- Raid-compatible architecture
- optional MiniCoin/community Strategy connection can be enabled later without changing the Cabinet ID

Community Raid concept:

three selected levels, ranked by:

1. completion validity
2. stars collected
3. total completion time

Association with Skrumpeys does not mean a fungible liquidity pair.

COINDLE

Community concept:

COIN / RWA

No reskin.

Coindle demonstrates:

GAME FIRST, ECONOMY LATER.

It remains live/playable without a MiniCoin or supported Community Asset economy.

The product should show an economy state such as:

MiniCoin Economy - Coming Soon

The existing Cabinet keeps the same `gameId` and leaderboard history when/if its economy becomes available later.

Genre:

daily deduction / guessing puzzle

Metrics:

- solved/not solved
- guesses
- completion time
- streak

Do not invent a synthetic COIN token or unsupported Monad asset.

SKIP TO 10K

Community concept:

ETH / WETH

No reskin.

Preserve the current game identity and deterministic infrastructure where useful.

Target:

- Cartridge integration
- Free Play
- normalized RunResult
- economy-compatible metadata/interface

Do not hardcode or deploy against an unverified bridged WETH path. The economy remains pending until the protocol pass verifies the exact asset/liquidity route.

Skip to 10K may later serve as the **stretch second MiniCoin deployment** after the primary Skrumpey Slide economic loop is stable.

HERO NEW SUBMISSION - SKRUMPEY SLIDE

Skrumpey Slide is **not** an initial seeded/published Cabinet.

It is intentionally withheld from the initial game-integration attack so it can prove the creator pipeline later:

```
SUBMITTED
  -> ADMIN APPROVED
  -> DEVELOPER DEPLOYS
  -> PUBLISHED
```

Genre:

physics launch / momentum steering / destruction-score arcade game

Community Asset:

Skrumpeys NFT collection

Competitive metrics can include:

- distance
- total score
- destruction
- collectibles

Skrumpey Slide should demonstrate:

- real Cartridge submission
- technical validation
- admin approval
- developer Deploy action
- Registry/publication transition
- Community Asset association
- optional MiniCoin deployment/economic layer
- Ranked Run
- Community Raid
- NFT community/entitlement integration

When it becomes published, the public Arcade should render it automatically from Registry state without homepage code changes.

---

8. THREE GAME MODES

Every compatible game can expose some or all of:

FREE PLAY

Requirements:

- instant
- no wallet
- unlimited retries
- no payment
- scores may remain local

Primary purpose:

FUN

RANKED RUN

Requirements:

- Player Account
- defined competitive rules
- optional entry payment
- verifiable run result
- persistent leaderboard/history
- Passport update

Primary purpose:

individual competition

COMMUNITY RAID

Requirements:

- defined game
- community
- objective
- start/end
- entry rules
- attempts
- leaderboard
- reward definition
- result settlement

Primary purpose:

community competition

---

9. GAME IMPORT / APPROVAL - V0

V0 is CURATED.

Do not build permissionless launching.

The v0 lifecycle is:

```
SUBMIT -> VALIDATE -> ADMIN APPROVE -> DEVELOPER DEPLOY -> PUBLISHED
```

Step 1 - Developer Submission

Developer submits a Token Arcade-compatible Cartridge plus launch metadata/configuration.

The initial legacy games may be seeded as already approved/deployed; new games should use the real pipeline.

Step 2 - Manifest

Each game provides metadata such as:

```
{
  id,
  name,
  description,
  version,
  artwork,
  developer,
  supportedModes,
  supportedObjectives,
  communityOptional,
  minicoinOptional
}
```

The Registry/publication record is Cabinet-specific. Reusing a Gameplay Skeleton for another community creates a different Cabinet/game ID and a different leaderboard by default.

Step 3 - Technical Validation

Check:

- builds
- mounts
- destroys cleanly
- result schema valid
- no broken imports
- mobile compatibility
- declared Score Verifier is supported

- deterministic seed behavior where the game/Verifier requires it
- no obvious malicious behavior
- no game-local wallet/economy logic leaking into the hosted Arcade path

Step 4 - Admin Review / Approval

Admin reviews:

- Cartridge validation
- metadata/artwork
- modes/objectives
- Community Asset association
- optional MiniCoin/economy request
- approved Strategy selections
- safety/compliance flags

Admin then approves or rejects.

Step 5 - Developer Deploy

After approval, the developer receives the Deploy action.

Deploy publishes the approved game/configuration through the v0 platform path. Where an onchain MiniCoin launch remains platform-authorized under the Protocol Source of Truth, the developer action invokes that approved deployment path rather than receiving unrestricted Factory authority.

Step 6 - Published

Registry state becomes deployed/published.

Frontend automatically displays the Cabinet from Registry state.

An existing game-only Cabinet can later enable an approved MiniCoin/economy without changing its `gameId` or resetting leaderboard history.

---

10. MINICOIN v0

A Minicoin is OPTIONAL.

Do not require every game to launch one.

For Metropolis, implement enough infrastructure to demonstrate:

Skrumpey Slide -> deploy Minicoin -> associate with Skrumpey Slide

Do not attempt to finish the final Token Arcade tokenomics.

V0 Minicoin requirements:

- standardized deployment
- metadata
- game association
- creator/admin
- Community Asset association
- strategy/config reference
- treasury reference
- clearly experimental

A `MinicoinFactory` is preferred over bespoke deployment code.

Do not promise:

- appreciation
- returns
- future `$TA`
- guaranteed rewards

---

11. TREASURY

No Token Arcade token exists.

Protocol-side revenue goes to:

Token Arcade Treasury

V0 treasury should be simple and auditable.

Fee configuration should support conceptual routing such as:

```
Entry
|-- Reward / Prize Pool
|-- Community
`-- Token Arcade Treasury
```

Do not hardcode percentages globally.

Make configuration explicit.

Treasury design should preserve future support for:

- MON
- stablecoins
- multiple chains
- different Community Strategies

---

12. SCORE VERIFICATION

Do NOT trust arbitrary scores supplied by the browser.

V0 uses a **game-specific Score Verifier** boundary.

The browser/game produces a structured run result, but Token Arcade decides how that result is accepted for Ranked/Raid use.

Supported verifier patterns include:

A. Session / telemetry validation

Authenticated run session plus sanity checks, timing/plausibility bounds, replay protection, and game-specific telemetry validation.

B. Server-authoritative verification

A trusted server/judge owns hidden or authoritative state and determines the accepted result.

C. Deterministic replay

The game supplies sufficient inputs/seed/version data for the verifier to replay the simulation and recompute the result.

Deterministic replay is desirable where practical, but **it is not a universal admission requirement for every Raid-ready Cartridge.**

Requirements:

- every Ranked/Raid-capable game declares its verifier type
- the client is never the final authority
- run IDs/session tokens are replay-protected
- verification is pinned to game/version/rules
- invalid/tampered results are rejected
- the strongest practical verifier should be used without rewriting a working game solely to satisfy replay for Metropolis

Contracts/backend accept only authenticated/verified settlement data.

Do not attempt to solve perfect anti-cheat for Metropolis.

Build a credible verification boundary that can harden later.

---

13. COMMUNITY RAID v0

Admin should be able to create a raid with:

```
{
  raidId,
  gameId,
  communityId,

  objective,

  seed?,

  startsAt,
  endsAt,

  maxAttempts,

  entryAsset?,
  entryAmount?,

  rewardType,
  rewardAsset?,

  communityShare?,
  treasuryShare?
}
```

Raid UI should clearly display:

- game
- community
- objective
- prize
- entry
- attempts
- time remaining
- leaderboard
- rules

For Skrumpeys:

Support a pre-deposited NFT prize through an escrow/reward vault.

Do NOT implement automated NFT floor purchasing for v0.

---

14. PLAYER PASSPORT v0

Minimum viable Passport:

- username/display name
- avatar/PFP
- Privy player identity
- linked ownership wallet
- games played
- Ranked Runs
- raids entered
- best scores
- wins/rewards
- communities
- Arcade Score placeholder/calculation if useful

NFT PFP:

Player should eventually be able to select an owned Community NFT.

For Metropolis, implement the cleanest reliable path possible.

Do not block the entire release on complex NFT browsing.

---

15. FRONTEND / UX

DESIGN NORTH STAR

The Internet Is Fun.

The product should feel:

- warm
- playful
- inviting
- nostalgic
- tactile
- slightly chaotic
- modern
- game-first
- internet-native

NOT:

- financial dashboard
- DEX interface
- terminal
- casino lobby
- generic Web3 gradient site

Crypto information should exist where useful without overpowering games.

UX RULES

Rule 1

Guest reaches playable game with as little friction as possible.

Rule 2

Do not ask for wallet/account until persistent value becomes relevant.

Rule 3

Every major action should have an obvious next step.

Rule 4

Keep blockchain status human-readable.

Prefer:

Entry confirmed

over:

Transaction 0x749... finalized

Advanced details can be expandable.

Rule 5

Make mode differences visually obvious.

Rule 6

Mobile browser experience matters.

Rule 7

Loading, empty, transaction, error and success states must be designed-not afterthoughts.

---

16. PRIMARY SCREENS

V0 should include:

Arcade Home

- game cards
- featured game
- active raid
- play buttons
- mode indicators
- simple account state

Game Detail

- artwork
- description
- Play Free
- Ranked Run
- Community Raid
- leaderboard
- Community Asset
- Minicoin if applicable

Game Player

Game-dominant interface.

Do not surround the game with unnecessary dashboard UI.

Raid Lobby

- raid rules
- prize
- community
- leaderboard
- attempts
- countdown
- enter/play CTA

Player Passport

- identity
- PFP
- scores
- games
- raid history
- achievements/rewards

Creator / Arcade Portal

- submit/register Cartridge

- configure metadata
- configure modes/objectives
- select Community Asset
- request optional MiniCoin/economy
- choose approved Strategy toggles
- view validation state
- view `submitted / approved / deployed` status
- Deploy after approval

Admin Review Console

- review submitted Cartridge/manifest
- inspect validation results
- approve/reject
- approve Community Asset/economy configuration
- manage publication/safety state
- create/administer raids where required

The public Arcade reads deployed/published Registry state.

---

17. METROPOLIS DEMO SCRIPT

This is the product acceptance test.

SCENE 1 - ARCADE EXISTS

Open Token Arcade.

Visible from Registry/publication state:

- eMONAD-branded Cabinet built from the Buy Bounce Skeleton
- Two Places at Once
- Coindle
- Skip to 10K

Play something immediately as Guest.

No wallet.

Coindle visibly demonstrates **Game First / Economy Later** with its economy unavailable/Coming Soon rather than blocking gameplay.

SCENE 2 - PLAYER ACCOUNT

Attempt Ranked Run on a supported Cabinet.

Token Arcade asks user to create/sign into Player Account.

Privy account/wallet created.

Player Passport becomes available.

Complete Ranked Run.

Result is accepted by that game's declared Score Verifier and persists to that Cabinet's leaderboard/history.

SCENE 3 - CREATOR SUBMITS SKRUMPEY SLIDE

Open Creator / Arcade Portal.

Skrumpey Slide appears as a **submitted** Cartridge with launch details, for example:

```
Community:
Skrumpeys

Community Asset Type:
ERC-721

Modes:
Free Play
Ranked Run
Community Raid

Objectives:
Distance
Total Score
Destruction

MiniCoin:
Enabled / requested
```

The game is not yet public.

SCENE 4 - ADMIN APPROVES

Switch to Token Arcade Admin state.

Review validation/configuration.

Click **APPROVE**.

Skrumpey Slide status becomes approved.

It is still not public until the developer deploys it.

SCENE 5 - DEVELOPER DEPLOYS

Return to Creator state.

Approved submission now exposes **DEPLOY**.

Developer initiates deployment.

Registry state becomes deployed/published.

Return to public Arcade.

Skrumpey Slide appears automatically without a homepage code change.

This proves the platform can ingest a new Cartridge.

SCENE 6 - SKRUMPEY SLIDE RANKED RUN

Play Free Play if useful for the demo, then enter Ranked Run.

V0 Ranked entry remains free unless the current economic/legal source explicitly changes that rule.

Play the scored run.

Result is verified through Skrumpey Slide's declared Score Verifier.

Cabinet leaderboard updates.

Passport updates.

SCENE 7 - COMMUNITY RAID / ECONOMY LOOP

Create or activate the Skrumpey Community Raid using the current Protocol Source of Truth.

Show the relevant v0 path:

- MiniCoin / WMON economy if deployed
- fees/accrual
- Prize Vault
- VaultThreshold trigger for Economy Raid
- manually deposited Skrumpey hero NFT if approved/available, otherwise a labeled stand-in
- Raid rules / attempts / objective

Player enters.

Score registers.

Leaderboard/attempt state updates.

Settlement/reward state is visible in human-readable UX.

SCENE 8 - CLOSE

The takeaway should be obvious without a 10-minute explanation:

> Token Arcade can host games, turn them into competitive experiences, connect them to existing communities, optionally attach programmable game economies, and onboard a new game through a real Submit -> Approve -> Deploy pipeline.

The initial games and Skrumpey Slide do not need to share mechanics, economies, or leaderboards to participate in the same Arcade platform.

---

18. CODEX WORKSTREAM / BRANCH PLAN

Branches may be renamed to fit repository convention.

`feat/v0-foundation`

Owner: Architecture agent

Checklist:

- inspect repository
- dependency audit
- route structure
- shared types
- environment config
- global error boundaries
- integration contracts/interfaces
- game cartridge interfaces
- document architecture

`feat/v0-design-system`

Owner: UI/UX agent

Checklist:

- typography
- spacing
- surfaces
- cards
- buttons
- inputs
- navigation
- responsive shell
- game card
- mode selector
- modal/drawer patterns
- transaction/status feedback
- empty/loading/error states
- mobile pass
- accessibility pass

North Star:

warm arcade-not crypto dashboard

`feat/v0-auth-privy`

Owner: Identity agent

Checklist:

- Guest state
- Privy login
- embedded wallet
- Player Account
- logout
- restore session
- account transition from Guest
- human-readable wallet UX

`feat/v0-ownership`

Owner: Identity/onchain agent

Checklist:

- external ownership wallet link
- signed association
- OwnershipResolver interface
- ERC-20 ownership check
- ERC-721 ownership check
- Delegate adapter if supported/reliable
- no asset transfers required
- eligibility API

`feat/v0-passport`

Owner: Product/backend agent

Checklist:

- profile
- display name
- PFP
- linked wallets
- games played
- scores
- Ranked history
- Raid history
- achievements/rewards placeholders
- community memberships
- clean Passport UI

`feat/v0-game-sdk`

Owner: Games platform agent

Checklist:

- cartridge interface
- manifest schema
- mode config
- objective schema
- run result schema
- lifecycle handling
- mounting/unmounting
- error isolation
- seed/config passing
- run completion event

`feat/v0-games-existing`

Owner: Game integration agent

Checklist:

eMONAD Cabinet from Buy Bounce source

- audit source repo
- establish Cartridge adapter
- Free Play
- Ranked result
- eMONAD skin/theme integration
- preserve scored mechanics unless intentionally versioned
- public Cabinet identity independent from the internal Buy Bounce Skeleton name

Two Places at Once

- Cartridge adapter
- Free Play
- timed Ranked result
- stars/collectibles
- Skrumpeys Community Asset metadata
- no broad reskin
- Raid-compatible level selection/result model where practical

Coindle

- Cartridge adapter
- Free Play/daily/archive logic
- guesses/time/streak result schema
- COIN/RWA concept metadata
- game-only / Economy Coming Soon state

Skip to 10K

- Cartridge adapter
- Free Play
- normalized RunResult
- ETH/WETH concept metadata
- no reskin
- no unverified asset/economy deployment

Do not rebuild games unnecessarily.

All four initial Cabinets must render from Registry/publication state rather than hardcoded homepage cards.

`feat/v0-skrumpey-slide`

Owner: dedicated game agent / later portal integration

Skrumpey Slide is intentionally **not part of the initial legacy Cartridge import**.

When ready, it should enter through the real Creator Portal:

- submitted
- validated
- admin approved
- developer deployed
- Registry published

Checklist after submission:

- use existing Skrumpey Slide brief
- working game
- Free Play
- Ranked-compatible
- Raid-compatible
- declared Score Verifier
- result output
- mobile controls
- polish
- asset attribution/source notes
- no wallet/economic logic inside the game

`feat/v0-game-registry`

Owner: Platform agent

Checklist:

- Game Registry data model
- stable Cabinet/game IDs
- source Skeleton metadata where useful
- `submitted / approved / deployed-published / paused` state
- mode support
- Community Asset association
- optional MiniCoin association
- Cabinet-specific leaderboard namespace
- frontend auto-populates from deployed/published Registry entries
- existing game can enable economy later without changing game ID/history

---

`feat/v0-admin-console`

Owner: Creator/Admin UX agent

Checklist:

- creator submission view
- admin allowlist/review state
- register/import Cartridge
- review manifest/validation
- configure modes/objectives/community/economy request
- admin approve/reject
- developer Deploy action after approval
- publish state derived from successful deployment
- pause/unpublish safety action
- create/administer raid

Optimize for demo clarity.

`feat/v0-contracts-core`

Owner: Smart contract agent

Checklist:

- identify minimum contract surface
- Game Registry if onchain requirement justified
- treasury routing
- authorization
- events
- deployment scripts
- Monad deployment configuration
- tests

Do not overengineer governance.

`feat/v0-minicoin`

Owner: Token contracts agent

Checklist:

- Minicoin implementation
- MinicoinFactory
- metadata

- game association
- Community Asset association
- strategy config
- treasury reference
- deployment UI connection
- tests
- no `$TA`

Do not implement speculative roadmap tokenomics unless already stable.

`feat/v0-community-strategy`

Owner: Protocol agent

Checklist:

- CommunityAsset type
- CommunityStrategy interface
- ERC20 strategy skeleton
- ERC721 strategy
- ownership eligibility
- community treasury config
- reward config
- keep NFT and fungible logic separate

`feat/v0-ranked-runs`

Owner: Gameplay/backend agent

Checklist:

- entry flow
- game config
- run session
- result capture
- result verification
- persistent result
- leaderboard
- Passport update
- failure/retry states

`feat/v0-raids`

Owner: Competition agent

Checklist:

- create raid
- start/end
- objectives
- attempts
- deterministic seeds
- leaderboard
- entry logic
- result submission
- settlement
- status states

`feat/v0-reward-vault`

Owner: Contracts agent

Checklist:

- ERC-721 prize deposit
- raid association
- winner settlement
- cancellation/refund path
- authorization
- events
- tests

No automated NFT purchasing.

`feat/v0-score-verification`

Owner: Security/backend agent

Checklist:

- run IDs
- session issuance
- result payload
- server/authority signature
- replay prevention
- leaderboard validation
- basic anti-tamper safeguards
- documentation of limitations

`feat/v0-metropolis-demo`

Owner: Integration/demo agent

Checklist:

- seed demo data
- initial three games
- admin demo account/state
- Skrumpey configuration
- Minicoin launch path
- sample raid
- demo-friendly balances
- resilient loading
- graceful fallback states
- exact demo script rehearsal

`feat/v0-security-qa`

Owner: QA/security agent

Checklist:

- contract tests
- permissions
- treasury routing
- prize custody
- verifier-specific tamper tests (session/server/replay as applicable)
- invalid signatures
- wallet state
- disconnected state
- mobile
- multiple browsers
- route cleanup
- game lifecycle
- asset loading
- build warnings
- TypeScript
- console errors
- transaction failure UX
- accessibility basics

`release/metropolis-v0`

Only merge features that improve the demo or preserve necessary architecture.

No roadmap experimentation on release branch.

---

19. BUILD ORDER

BLOCKER LAYER

Build first:

1. foundation
2. design system
3. game SDK
4. auth/account model
5. shared protocol types

PARALLEL LAYER

Then parallelize:

- existing games
- Passport
- Game Registry
- Admin Console
- ownership resolver
- creator/admin publication pipeline
- core contracts

ECONOMIC / COMPETITION LAYER

Then:

- Minicoin
- Community Strategy
- Ranked Runs
- Raids
- Reward Vault
- score verification

FINAL LAYER

Then:

- full integration
- QA/security
- design polish
- mobile polish
- Metropolis demo state
- deploy

---

20. CUT ORDER IF DEADLINE GETS DANGEROUS

Do NOT sacrifice the end-to-end demo to save optional systems.

Cut in this order:

1. fancy Passport achievements
2. Delegate integration - retain direct external wallet ownership proof
3. advanced ERC20 Community Strategy
4. complex Minicoin economics
5. elaborate cosmetics
6. extra game modes
7. nonessential game polish
8. extra onchain writes

DO NOT CUT:

- playable arcade
- Guest flow
- Privy Player Account
- three-mode architecture
- Game Registry
- Submit -> Admin Approve -> Developer Deploy publication flow
- Skrumpey Slide portal launch
- Minicoin deployment demonstration
- Ranked Run
- Community Raid
- result verification boundary
- leaderboard
- treasury routing
- reward settlement proof

---

21. ROADMAP - NOT METROPOLIS

Everything below is deliberately outside v0 unless required for extensibility.

PHASE 1 - CURATED CREATOR SUBMISSIONS

External developers submit Minigames/Cartridges.

Token Arcade team:

- reviews
- audits
- security checks
- validates Cartridge compatibility
- reviews token/reward behavior
- approves or rejects the submission

After approval, the **developer initiates Deploy** through the approved platform path.

Think:

App Store / arcade curator stage.

The initial v0 version is curated; later versions progressively automate the checks and remove human approval for safe/audited templates.

PHASE 2 - CREATOR LAUNCH TOOLING

Creators receive structured tools to configure:

- game
- art/lore
- game template
- Community Asset
- Minicoin
- reward model
- raid format
- hooks
- treasury
- cosmetics

Still curated.

PHASE 3 - AI GAME LAUNCHPAD

Prompt-assisted creation.

Experience should feel closer to:

Mad Libs + game creator + token/community configuration

than:

blank AI chat box

Creator selects:

- original IP / lore

- character
- art style
- 2D comic
- 2D pixel sprite
- 3D asset
- game skeleton
- Single Player
- Multiplayer
- turn-based
- simultaneous arena/.io
- competitive metric
- rewards
- Community Asset
- Minicoin strategy
- raid structure

AI generates against audited game skeletons.

PHASE 4 - COMMUNITY ARCADES

A Community Asset can develop its own arcade.

Example:

Skrumpeys Arcade

Multiple games.

Multiple Minicoins.

Shared player community.

Shared rewards.

Shared raids.

This begins transforming Token Arcade from:

one arcade

into:

infrastructure for internet communities to have arcades.

PHASE 5 - CLANS / COMMUNITIES / SPRINTS

Persistent collective competition.

Influences:

- clan wars
- community coins
- friend.tech-style community identity
- social speculation
- arcade seasons

Players join or represent communities.

Individual gameplay contributes to collective score.

Longer events become:

Community Sprints

Potential inputs:

- scores
- wins
- participation
- holder count
- volume
- Minicoin activity
- treasury
- Community Asset metrics

Rewards become larger and collective.

PHASE 6 - AUTOMATED RAID TRIGGERS

Community Raid activation can eventually happen through two paths.

ORGANIC / SYSTEM-TRIGGERED

A Minicoin or Community Strategy accumulates enough activity/value to automatically fund/activate an event.

Possible trigger inputs:

- LP depth
- holder count
- Minicoin market cap
- Community Asset market cap
- volume
- volume spike
- treasury size
- fees accumulated
- participation
- combination score

The system should optimize for:

sustainable + fun

not simply:

number go up

COMMUNITY-FUNDED

A community or individual manually funds a raid using:

- Community Asset
- NFT
- stablecoin
- MON/ETH/SOL
- Minicoin
- other configured reward

They select:

- game
- objective
- rules
- duration
- attempts
- reward

And launch a community event.

---

22. PERSONALIZATION / COSMETICS ECONOMY

Games do not need Minicoins forever.

Token Arcade itself can monetize useful/fun identity features.

Potential store currency:

USDC / supported stablecoin

Potential cosmetics:

- profile borders
- card frames
- UI themes
- calling cards
- animated calling cards

- colored usernames
- badges
- titles
- emotes
- avatars
- character skins
- game-specific cosmetics
- cross-game characters/items

Think:

MW2 calling cards + old internet profiles + arcade identity

Player-owned elements should increasingly persist across games.

---

23. FOUNDING GAMER / NFT POSSIBILITY

No commitment.

Potential future path:

Token Arcade launches **no fungible protocol token**.

Instead, early access/deployment rights could potentially belong to a limited Founding Gamer / Founding Creator NFT collection.

Potential utility:

- beta testing
- early creator tools
- cartridge access
- game deployment rights
- community tooling
- governance experiments
- launch access

Potential deployment partner/mechanism can be evaluated later.

Do not build or promise this in v0.

---

24. PERMISSIONLESS ENDGAME

Long-term objective:

Creator can safely launch a Minigame/Minicoin without Token Arcade staff manually approving every deployment.

To reach that state Token Arcade would need:

- constrained audited game skeletons
- constrained contract templates
- automated contract validation
- static analysis
- AI-assisted security review
- simulation
- economic guardrails
- permissions
- standardized hooks
- malicious-code isolation
- reputation
- monitoring
- circuit breakers

Only then does:

> "Anyone can launch a game economy"

become credible.

This is a roadmap destination-not a hackathon feature.

---

25. MULTICHAIN

Do NOT define Token Arcade as "a Monad arcade."

Define:

> Token Arcade v0 launches on Monad.

Preserve the possibility of future chain adapters.

Conceptually:

```
Token Arcade
    |
Game / Player / Raid Layer
    |
Settlement Adapter
    |
|-- Monad
|-- Ethereum
|-- Robinhood Chain
|-- Solana
`-- future
```

Do not build unused adapters today.

Simply avoid spreading Monad-specific calls throughout game code.

---

26. OPTIONAL FUTURE TOKEN A

A future `$TA` is explicitly undecided.

Reasons to defer:

- tokenomics require testing
- treasury behavior requires testing
- Minicoin behavior requires testing
- community incentives require testing
- chain strategy may change
- funding model may change
- Token Arcade may not need a fungible protocol token at all

Possible future capital/community models include:

- no protocol token
- `$TA`
- equity/fundraise
- NFT membership
- creator licenses
- stablecoin revenue
- hybrid

Do not architect v0 around needing one answer.

---

27. META / INSPIRATION STACK

Token Arcade is deliberately compressing several internet/crypto/game traditions.

The synthesis is more important than any individual reference.

NOSTALGIA

- Flash games
- Miniclip
- browser arcades
- Newgrounds-era immediacy
- old web profiles
- calling cards
- local/high-score competition

GAMING

- 2D illustrated games
- 2D pixel games
- 3D characters/assets
- single player
- multiplayer
- turn-taking competition
- simultaneous `.io` competition
- arcade score attack
- platformers
- physics games
- daily puzzles
- clan wars

CRYPTO

- community coins
- NFTs
- memetics
- reward tokens
- tax tokens
- liquidity
- hooks
- treasury systems
- token-gated communities
- onchain identity
- permissionless deployment

CREATOR / AI

- AI-assisted development
- prompt engineering
- game templates
- reusable game skeletons
- original IP
- lore
- community remixing
- generative content

SOCIAL

- fandom
- clans
- communities
- collective score
- competition

- status
- speculation
- ownership
- rewards

The design objective is NOT:

> Put all these features on one screen.

It is:

> Compress these ideas into an experience where playing an internet game naturally creates community activity.

---

28. INSPIRATION NOTEBOOK

Existing references mentioned during ideation include concepts/projects such as:

- Miniclip
- FWA
- Claus
- IMD
- NetNet
- STONK
- TokenStrategy
- Uniswap
- Hooked
- FrenPet
- NFTs
- tax coins
- browser games
- community coins
- `.io` games

These are references/inspirations, NOT dependencies.

Do not infer technical requirements from a name in this list without an explicit verified specification.

---

29. THE CORE FLYWHEEL

V0 should demonstrate the earliest version of this:

```
FUN GAME
   ↓
PLAYER ATTENTION
   ↓
PLAYER IDENTITY
   ↓
COMPETITION
   ↓
COMMUNITY
   ↓
ECONOMIC ACTIVITY
   ↓
REWARDS / TREASURY
   ↓
MORE EVENTS / MORE GAMES
   ↓
MORE PLAY
```

Minicoins can amplify the loop.

Community Assets give the loop an existing audience.

Raids coordinate attention.

Player Passport preserves history.

AI eventually increases game supply.

Permissionless deployment eventually increases creator supply.

None of those systems matter if the first box-

FUN GAME

-is bad.

---

30. V0 SUCCESS CRITERIA

Metropolis v0 succeeds if a stranger can watch the demo and understand:

1. Token Arcade already contains playable Cabinets from Registry state.
2. They can play without connecting a wallet.
3. They can create a persistent arcade identity.
4. Games support different competitive modes.
5. A Gameplay Skeleton can be adapted into a separately branded Cabinet without making its leaderboard the Skeleton's leaderboard.
6. A developer can submit a new Cartridge.
7. Token Arcade Admin can approve it.

8. The developer can Deploy the approved game and it appears automatically in the public Arcade.
9. A game can exist without a MiniCoin and add an economy later without changing its game ID/history.
10. A new game can optionally connect to a Community Asset and MiniCoin/Strategy configuration.
11. Players create verifiable results through a game-appropriate Score Verifier.
12. Rewards/fees can move through programmable infrastructure.
13. The product feels like an arcade - not a DeFi application.
14. The architecture obviously has room to grow beyond this hackathon.

If those things are true:

STOP.

Token Arcade v0 is complete.

Do not sacrifice the finished Metropolis build to implement the future.

FINAL BUILD MANTRA

The Internet Is Fun.

Make the games fun.

Make joining effortless.

Make wallets disappear until useful.

Make identity persistent.

Make competition obvious.

Make communities feel represented.

Make rewards understandable.

Make launching a new game feel magical.

Keep the infrastructure modular.

Keep the economics experimental.

Do not launch `$TA`.

Do not build the entire roadmap.

Ship the arcade.